

에이전트와의 대화 상태 유지하기,
LangGraph_CheckPointer

에이전트와의 대화 상태 유지하기, LangGraph_CheckPointer

1. 랭그래프와 메모리
2. 체크포인트 설정
3. 장기 기억을 위한 메모리 최적화

1. 랭그래프와 메모리

- LLM을 두뇌로 사용하는 에이전트도 기본적으로 상태를 저장하지 않는(stateless) 특성이 있음
- 사용자와의 대화 맥락(context)을 기억하고, 그것을 바탕으로 소통할 수 있어야 함
- 이번 장에서는 랭그래프(LangGraph)의 핵심 기능인 체크포인트를 활용해 에이전트에게 기억력을 부여하는 방법을 다룸
- 기존 선형적 체인 구조의 한계와 이를 극복하기 위해 등장한 랭그래프의 구조적 특징을 다룸
- 에이전트의 반복적인 추론 과정을 구현하기 위해 왜 그래프 방식이 필수적인지 알아보고, 랭그래프의 핵심 개념인 ‘상태’가 어떻게 정의되고 관리되는지 살펴봄

1. 랭그래프와 메모리

● 왜 체인이 아니라 그래프인가?

- 랭체인이 버전 1.0으로 넘어오면서 크게 변화한 부분 중 하나는 바로 **에이전트의 상태 관리**
- 기존 랭체인이 에이전트의 ‘로직(어떤 도구를 쓸 것인가)’을 정의하는데 집중했다면, 랭그래프는 에이전트의 ‘상태(지금 어떤 단계이며, 이전 데이터는 무엇인가)’를 관리하는 프레임워크
- 기존의 랭체인은 이름 그대로 ‘체인’ 구조를 기본으로 했음. 체인은 A 단계가 끝나면 B 단계로, B 단계가 끝나면 C 단계로 넘어가는 선형적인(linear) 흐름을 가짐
- 이는 문서 요약이나 단순 질의 응답처럼 시작과 끝이 명확한 작업에는 효율적
- 하지만, 에이전트의 추론 과정은 선형적이지 않고 순환적(cyclic)임
- 기존의 체인 구조(비순환 그래프)로 이러한 ‘반복 루프’를 구현하는 것은 매우 복잡하고 코드가 난해해지는 문제가 있었음
- 랭그래프는 이러한 한계를 극복하기 위해 등장
- 랭그래프는 에이전트의 흐름을 **노드(node)**와 **엣지(edge)**로 구성된 그래프(graph)로 정의함
- 이를 통해 에이전트는 특정 조건이 만족될 때까지 자유롭게 이전 단계로 되돌아가거나, **상태**를 유지하며 계속해서 사고를 확장할 수 있게 됨
- 랭체인이 ‘절차 지향적’이라면, 랭그래프는 ‘상태 지향적’임

1. 랭그래프와 메모리

- 랭그래프의 핵심 구성 요소와 메모리 시스템

- 랭그래프의 핵심 개념 세 가지

1. 상태: 그래프 전체가 공유하는 ‘메모리’. 현재까지의 대화 내용, 도구의 실행 결과, 에이전트의 다음 행동 계획 등이 모두 이 상태에 저장됨
2. 노드: 실제로 작업을 수행하는 작업자. LLM이 될 수도 있고, 특정 파이썬 함수(도구)가 될 수도 있음. 각 노드는 작업을 마친 후 상태를 업데이트함
3. 엣지: 노드와 노드 사이를 연결하는 선. 다음 단계로 어디로 갈지 결정하는 ‘제어 흐름’을 담당. 여기서 ‘결과가 부족하면 다시 검색 노드로 돌아가라’는 식의 순환 구조가 만들어짐

- 이 장에서 다룰 메모리 기능은 바로 이 ‘상태’를 어떻게 저장하고 불러올 것인가에 대한 이야기임

- 체크포인트(checkpointer)

- 랭그래프에서 메모리 기능을 담당하는 핵심 컴포넌트
 - 에이전트가 행동을 취할 때마다 그 시점의 상태를 저장소에 저장하는 역할
 - 기존의 에이전트는 사용자 입력에 대해 에이전트가 실행되고 답변이 끝나면 모든 입출력 정보가 삭제됨
 - 여기에 체크포인트를 적용하면 입출력 정보 삭제 대신 ‘상태 저장’ 과정이 추가됨
 - 다음 질문이 시작되면 ‘상태 복구’ 과정을 통해 이전에 저장해 둔 지점에서부터 다시 대화를 이어갈 수 있게 됨
 - 이를 통해 에이전트는 마치 인간처럼 이전 대화를 기억하고 그 맥락 속에서 새로운 질문에 답할 수 있게 됨

1. 랭그래프와 메모리

- 랭그래프의 핵심 구성 요소와 메모리 시스템

- 체크포인트의 동작 프로세스

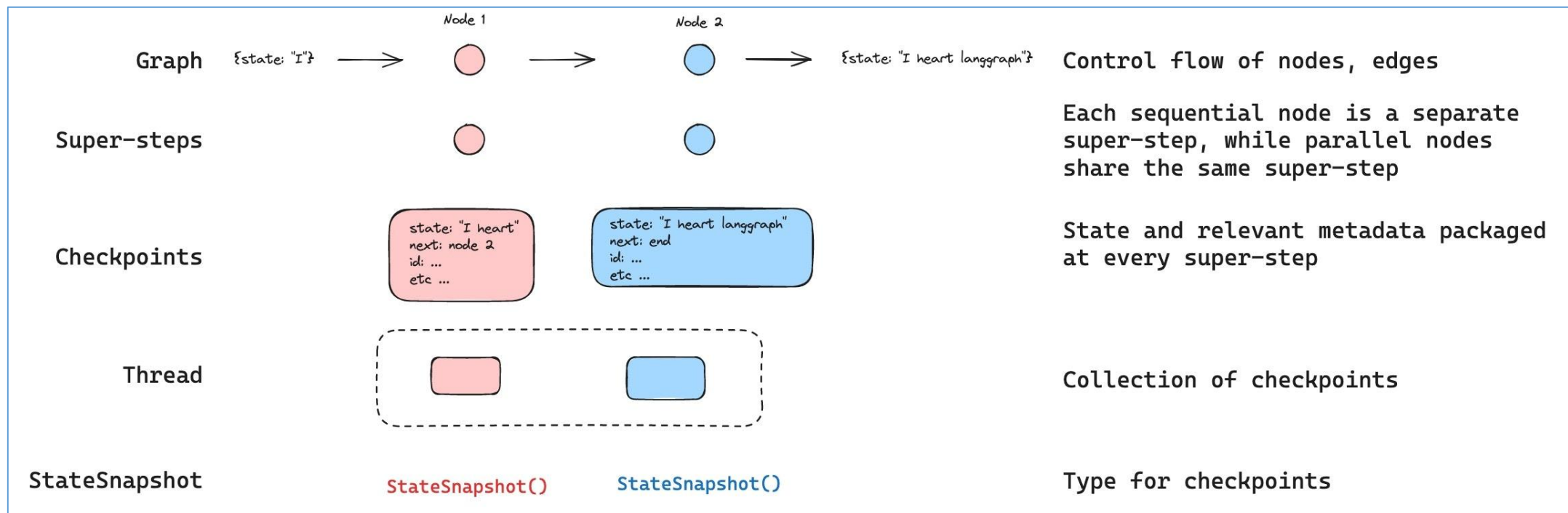
1. 스냅샷 저장: 사용자의 질문이 들어오거나 에이전트가 도구를 사용할 때마다, 현재의 모든 변수(대화 기록 등)를 데이터베이스에 저장함
2. 식별자: 수많은 사용자의 기록이 섞이지 않도록 thread_id라는 고유한 키를 사용하여 저장 구역을 나눔
3. 자동 복구: 동일한 thread_id로 접근하면 체크포인트는 가장 마지막에 저장된 상태를 불러와 에이전트에게 주입함. 덕분에 에이전트는 ‘아, 우리가 여기까지 이야기했었지’라고 인식하며 대화를 이어갈 수 있음

- 이때 중요한 개념이 바로 thread_id임
- 수많은 사용자가 동시에 에이전트를 사용할 때, 누구의 기억을 불러와야 할지 구분하는 식별자 역할
- ‘10.2.2절의 사용자별 대화 관리’의 실습에서 사용해 보았던 config의 ‘session_id’와 동일한 역할
- 랭체인 공식 문서의 예시에서는 ‘thread_id’를 채택하고 있을 뿐, 딥서너리의 키 값은 여러분들이 구현하고자 하는 서비스 상황에 맞게 재구성하면 됨

1. 랭그래프와 메모리

- 랭그래프의 핵심 구성 요소와 메모리 시스템

- 랭그래프를 활용한 Persistence 예시(출처: <https://docs.langchain.com/oss/python/langgraph/persistence>)



1. 랭그래프와 메모리

● RunnableWithMessageHistory vs. Checkpointer

1. 구조적 차이

- RunnableWithMessageHistory: 기존의 체인이 객체를 외부에서 감싸는 래퍼(wrapper) 형태임. 체인이 실행될 때마다 외부에서 ‘이전 대화 기록’을 끼워 넣는 방식. 체인 자체는 자신이 기억을 가지고 있는지 모르므로, 이렇게 누군가가 끼워 넣지 않으며 전부 사라짐
- Checkpointer: 랭그래프 에이전트의 실행 엔진(Runtime) 내부에 통합되어 있음. 인간의 뇌 해마처럼 에이전트가 생각하고 행동하는 모든 순간을 자동으로 기록. 에이전트 생성 시점에 한 번만 장착해 주면 스스로 기억을 관리

2. 저장하는 대상

- RunnableWithMessageHistory: 오직 대화 목록만 저장함. ‘사용자가 A라고 했고, AI가 B라고 했다’는 텍스트 기록이 전부
- Checkpointer: 전체 실행 상태를 저장함. 대화 목록뿐만 아니라, 현재 에이전트가 어떤 도구를 선택했는지, 도구의 실행 결과가 무엇인지, 에이전트가 다음에 수행할 단계가 어디인지, 내부 변수 값들은 무엇인지 등의 정보가 모두 포함됨

3. 관리의 편의성

- RunnableWithMessageHistory: 개발자가 매번 세션 ID를 관리하고, 체인을 호출할 때마다 config에 세션 정보를 담아서 수동으로 넘겨줘야 하는 번거로움이 있음
- Checkpointer: 고정된 식별자 하나만 있으면 됨. 랭그래프 엔진이 알아서 데이터베이스를 뒤져 해당 스레드의 마지막 상태를 찾아내고, 에이전트의 메모리에 물을 붓듯(hydration) 상태를 복구시킴

2. 체크포인트 설정

- 에이전트의 기억을 저장할 공간이 필요
- 랭그래프는 개발 상황에 맞춰 다양한 종류의 체크포인트를 제공함
- 그중 가장 대표적인 두 가지, InMemorySaver와 SqliteSaver를 단계별로 실습

● InMemorySaver로 메모리 구현하기

- 데이터를 컴퓨터의 메모리에 저장하는 방식
- 별도의 데이터베이스 설정이나 파일 생성이 필요 없어 설정이 매우 간편하고 속도가 빠름
- 코드를 작성하고 즉시 기능을 검증해야 하는 개발 초기 단계나 프로토타입을 만들 때 가장 적합
- 단, 메모리에 저장되는 만큼 프로그램을 종료하면 모든 기억이 사라짐을 명심

2. 체크포인트 설정

- InMemorySaver로 메모리 구현하기

- 기본 환경 설정

```
# 가상 환경 생성
$ python -m venv .venv

# 윈도우 가상 환경 활성화
$ source .venv/Scripts/activate # 맥 환경에서는 Scripts를 bin으로 변경

(.venv)
$ python -m pip install --upgrade pip

# 패키지 설치
(.venv)
$ pip install langchain langchain-openai langgraph dotenv langgraph-checkpoint langgraph-checkpoint-sqlite
```

- 앞서 langchain_chroma 패키지를 설치하며 Chroma가 의존하고 있던 dotenv 패키지가 함께 설치됨. 그러나 이번 실습 환경에서는 langchain_chroma를 사용하지 않으므로 dotenv 패키지를 반드시 직접 설치해야 함

- 오픈AI 키 등록

- 에이전트에 저장소를 장착하고자 한다면, 먼저 에이전트의 두뇌부터 결정
 - 오픈AI LLM 모델을 사용하기 위해 오픈AI API 키를 '.env' 파일에 작성하여 프로젝트 폴더에 함께 구성

2. 체크포인트 설정

- InMemorySaver로 메모리 구현하기

- 모델 등록

- 'in_memory_agent.py' 파일을 생성하여 모델 객체를 생성

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model="gpt-5-nano", temperature=0)
```

2. 체크포인트 설정

- InMemorySaver로 메모리 구현하기

- 체크포인트와 에이전트 생성
 - 이번 실습의 핵심은 ‘기억’ 할 수 있는 에이전트이므로, 특별한 도구나 시스템 프롬프트 없이 에이전트를 구성
 - 에이전트는 이전과 같이 create_agent 함수를 통해 생성할 수 있음
 - 체크포인트는 InMemorySaver 클래스로 생성할 수 있음

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent

... ( 생략 ) ...

checkpointer = InMemorySaver()

agent = create_agent(
    model=model,
    checkpointer=checkpointer
)
```

- 랭체인 1.x 이후 버전의 핵심은 간결함. 상태 유지, 메모리 관리 같은 복잡한 로직은 내부에 숨기고, 개발자는 핵심 로직 구현에만 집중할 수 있도록 설계됨
- 체크포인트 등록이 단 두 줄만으로 구성되는 것이 바로 이 철학을 보여주는 좋은 예

2. 체크포인트 설정

● InMemorySaver로 메모리 구현하기

- 체크포인트와 에이전트 생성
 - 이렇게 등록한 체크포인트를 사용하기 위해서는 지켜야 할 규칙이 있음
 - 에이전트에게 사용자 질문을 전달할 때, '누가 보낸 질문'인지를 '정해진 규칙에 따라' 알려주어야 함
 - 이를 위해 잠시, invoke 메서드의 구조를 살펴보겠습니다

```
(method)
def invoke(
    input: _InputAgentState | Command | None,
    config: RunnableConfig | None = None,
    *,
    context: Any | None = None,
    stream_mode: Literal['values'] = ...,
    print_mode: StreamMode | Sequence[StreamMode] = (),
    output_keys: str | Sequence[str] | None = None,
    interrupt_before: Sequence[str] | Literal['*'] | None = None,
    interrupt_after: Sequence[str] | Literal['*'] | None = None,
    durability: Durability | None = None,
```

- 두 번째 인자 config는 RunnableConfig 객체 형태로 전달되어야 함
- 체크포인트를 위한 config는 {"configurable": {"thread_id": "1"}} 와 같이 작성

2. 체크포인트 설정

- InMemorySaver로 메모리 구현하기

- 대화 기억 테스트
 - 체크포인트가 적용된 에이전트 호출 방법을 숙지했으니 이제 에이전트가 정말로 대화를 기억하는지 확인

```
# 사용자 식별을 위한 thread_id 설정
user_id = "user_123"
config = {"configurable": {"thread_id": user_id}}

# 1. 첫 번째 대화: 이름 정보 제공
query1 = "내 이름은 철수야"
print(f"사용자: {query1}")

result = agent.invoke(
    {"messages": [{"role": "user", "content": query1}]},
    config
)
print(f"Agent: {result['messages'][-1].content}")

# 2. 두 번째 대화: 기억력 테스트 (같은 thread_id 사용)
query2 = "내 이름이 뭐라고 했어?"
print(f"사용자: {query2}")

result = agent.invoke(
    {"messages": [{"role": "user", "content": query2}]},
    config
)
print(f"Agent: {result['messages'][-1].content}")
```

2. 체크포인트 설정

- InMemorySaver로 메모리 구현하기

- 출력 결과 확인

사용자: 내 이름은 철수야

Agent: 반가워요, 철수야! 무엇을 도와줄까? 필요한 게 있으면 말해줘.

사용자: 내 이름이 뭐라고 했어?

Agent: 당신의 이름은 철수라고 했어요. 더 도와드릴 게 있을까요?

- 에이전트가 “철수”라는 이름을 기억하고 답변하는 것을 볼 수 있음
- 여기서 핵심은 thread_id. 에이전트는 메모리에 저장된 상태들 중, 현재 질문과 함께 전달된 config의 thread_id 값이 동일한 상태를 찾아 맥락을 이해함
- 이 thread_id만 일치한다면 언제든지 동일한 대화 맥락을 불러올 수 있음
- 역으로, user마다 서로 다른 thread_id를 할당하기만 하면 각자의 독립된 대화 맥락이 자동으로 유지되는 것임

2. 체크포인트 설정

- InMemorySaver로 메모리 구현하기

- 새로운 유저와 대화

```
# 3. 세 번째 대화: 다른 사용자로 설정 (다른 thread_id 사용)
new_user_id = "user_789"
new_config = {"configurable": {"thread_id": new_user_id}}

query3 = "내 이름이 뭐라고 했어?"
print(f"사용자: {query3}")

result = agent.invoke(
    {"messages": [{"role": "user", "content": query3}]},
    new_config
)
print(f"Agent: {result['messages'][-1].content}")
```

- 출력 결과 확인

사용자: 내 이름은 철수야

Agent: 반가워요, 철수님. 무엇을 도와드릴까요? 편하신 호칭이나 대화 방식이 있으면 알려주세요. 필요한 것이 있으면 말씀해 주세요.

사용자: 내 이름이 뭐라고 했어?

Agent: 철수라고 하셨어요. 다음에도 그렇게 불러도 될까요? 필요하시면 다른 호칭으로도 부를 수 있어요.

사용자: 내 이름이 뭐라고 했어?

Agent: 지금 이 대화에서 당신의 이름을 아직 받지 못했어요. 이름을 알려주시면 이 대화에서 불러드리고 기억해둘게요. 단, 대화가 끝나면 기억도 지워집니다.
당신의 이름이 뭐예요?

2. 체크포인트 설정

- SqliteSaver로 메모리 영구화하기

- SQLite는 서버가 필요 없는 파일 기반의 데이터베이스
- 파이썬에 기본적으로 내장되어 있어 별도의 설치 과정도 필요 없음. '앞서 Chroma 벡터 저장소 만들고 저장하기' 절에서 특별한 설정 없이 벡터 저장소를 만들고 사용할 수 있었던 이유
- SQLite는 데이터베이스 전체가 단 하나의 파일(.db)로 관리되므로 이동과 관리가 매우 간편
- 이러한 특징 덕분에 가벼운 웹 서비스나 모바일 앱, 그리고 지금 우리가 만들고 있는 에이전트의 로컬 기억 저장소로 사용하기에 최적의 도구
- SqliteSaver를 사용하여 이 SQLite 데이터베이스에 에이전트의 기억을 영구적으로 저장해 보겠음

2. 체크포인트 설정

- SqliteSaver로 메모리 영구화하기
 - 파일 생성 및 모델 등록
 - 'sqlite_agent.py' 파일에 코드 작성

```
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()
model = ChatOpenAI(model="gpt-5-nano", temperature=0)
```

2. 체크포인트 설정

- SqliteSaver로 메모리 영구화하기

- 데이터베이스 연결과 체크포인트 초기화
 - SqliteSaver를 사용하려면 연결할 SQLite 데이터베이스가 필요
 - SQLite 데이터베이스는 파이썬 내장 모듈 sqlite3를 사용하여 조작할 수 있음

```
from langgraph.checkpoint.sqlite import SqliteSaver
import sqlite3

# 데이터베이스 연결 및 SqliteSaver 초기화
conn = sqlite3.connect("memory.db", check_same_thread=False)
checkpointer = SqliteSaver(conn)
```

- 데이터베이스에 연결하기 위해서 sqlite3 모듈의 connect 메서드를 사용
- 일반적으로는 연결 대상 데이터베이스의 경로를 첫 번째 인자로 작성하면 되지만, 체크포인트로 사용하려면 반드시 check_same_thread 인자를 False로 설정하여야 함
- SQLite는 기본적으로 같은 스레드에서만 연결을 사용하도록 강제
- 따라서, 다른 스레드에서 이 연결을 사용하려고 하면 에러가 발생
- 랭그래프의 에이전트는 비동기 처리와 멀티스레드 환경을 자주 사용
- 이때 SQLite가 스레드 간 연결 공유를 막으면 작동하지 않음. 이러한 상황을 방지하기 위하여 check_same_thread=False를 사용
- 데이터베이스 연결을 마쳤다면, SqliteSaver에게 연결 정보를 넘겨주기만 하면 체크포인트 초기화는 끝

2. 체크포인트 설정

- SqliteSaver로 메모리 영구화하기

- 에이전트 생성과 코드 실행

```
agent = create_agent(  
    model=model,  
    checkpointer=checkpointer  
)  
  
# 사용자 식별을 위한 thread_id 설정  
user_id = "user_123"  
config = {"configurable": {"thread_id": user_id}}  
  
... ( 생략 ) ...  
  
# 3. 세 번째 대화: 다른 사용자로 설정 (다른 thread_id 사용)  
new_user_id = "user_789"  
new_config = {"configurable": {"thread_id": new_user_id}}  
  
query3 = "내 이름이 뭐라고 했어?"  
print(f"사용자: {query3}")  
  
result = agent.invoke(  
    {"messages": [{"role": "user", "content": query3}]},  
    new_config  
)  
print(f"Agent: {result['messages'][-1].content}")
```

- 에이전트 생성과 초기 설정은 앞선 'InMemorySaver로 메모리 구현하기' 절에서 작성했던 것과 동일

2. 체크포인트 설정

- SqliteSaver로 메모리 영구화하기

- 출력 결과 확인

사용자: 내 이름은 철수야

Agent: 네, 철수님 맞습니다. 오늘 어떤 도움을 드릴까요? 필요하신 주제나 작업을 말씀해 주시면 바로 도와드리겠습니다.

사용자: 내 이름이 뭐라고 했어?

Agent: 철수라고 하셨어요. 필요하신 게 있으면 말씀해 주세요.

사용자: 내 이름이 뭐라고 했어?

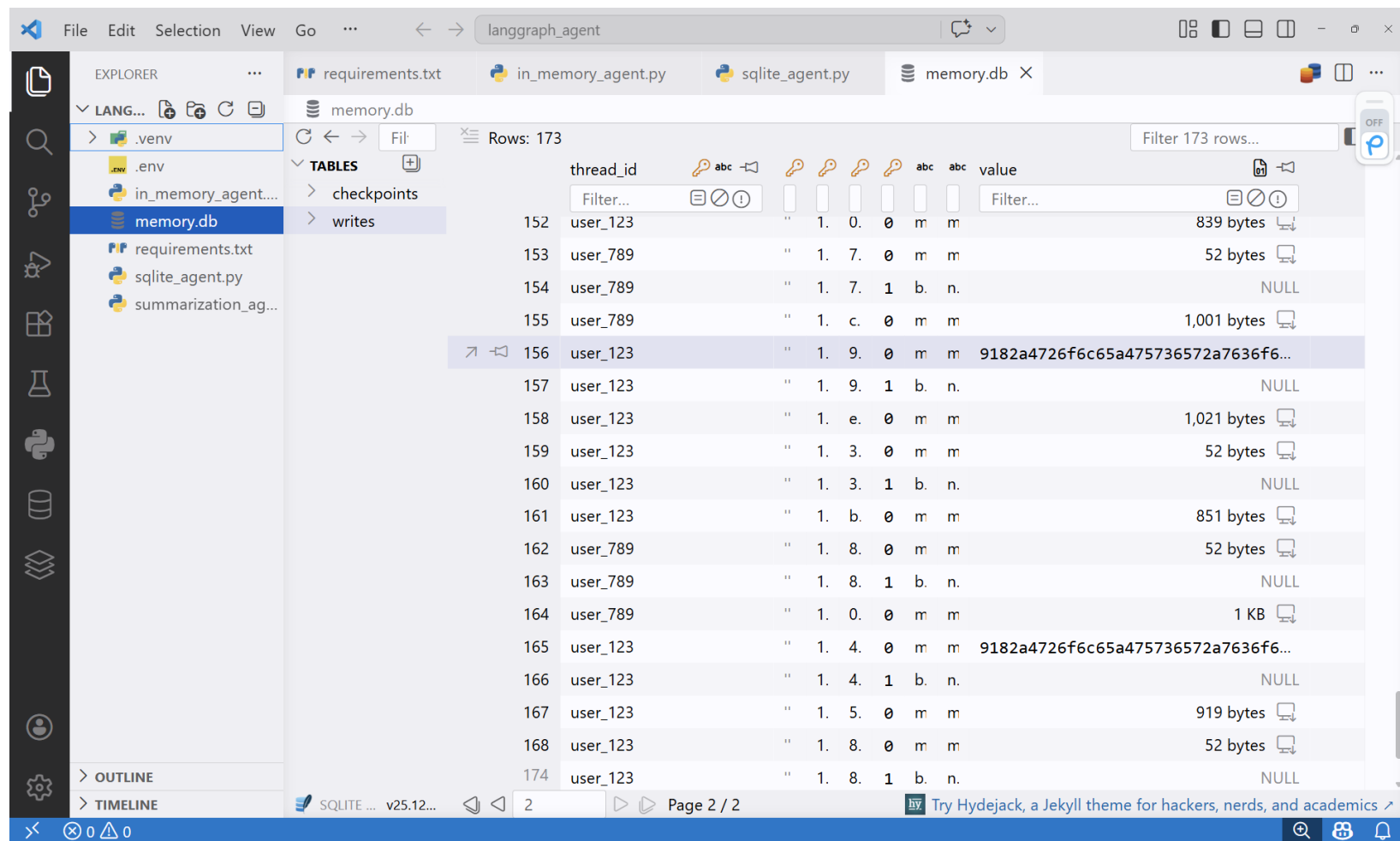
Agent: 지금 이 대화에서 당신의 이름을 아직 듣지 못했어요. 어떤 이름이나 호칭으로 불러드릴지 알려주시면 그 이름으로 불러드릴게요. 예) "OO님", "손님" 또는 원하시는 다른 이름이 있나요? 알려주시면 바로 사용하겠습니다.

- InMemorySaver를 사용할 때와 같이 thread_id 값이 무엇이었는지에 따라 이름을 기억하고 있거나, 알지 못하다는 답변을 내놓음
- 상태의 저장과 복구, 식별자에 따른 구분을 잘 수행하고 있음
- SqliteSaver의 핵심은 데이터베이스에 저장되어 영구적으로 보존된다는 점
- 작업 중인 폴더의 파일 목록에 'memory.db'가 생성
- 각 테이블에서는 'thread_id' 컬럼으로 어떤 사용자와의 대화 내용인지 잘 구분하여 기록하고 있음

2. 체크포인트 설정

- SqliteSaver로 메모리 영구화하기

- 출력 결과 확인



The screenshot shows a SQLite database named 'memory.db' with 173 rows. The 'writes' table is selected, showing columns for thread_id, value, and size. The data includes user_123 and user_789 with various values and sizes.

thread_id	value	size
152 user_123	"	839 bytes
153 user_789	"	52 bytes
154 user_789	"	NULL
155 user_789	"	1,001 bytes
156 user_123	"	9182a4726f6c65a475736572a7636f6...
157 user_123	"	NULL
158 user_123	"	1,021 bytes
159 user_123	"	52 bytes
160 user_123	"	NULL
161 user_123	"	851 bytes
162 user_789	"	52 bytes
163 user_789	"	NULL
164 user_789	"	1 KB
165 user_123	"	9182a4726f6c65a475736572a7636f6...
166 user_123	"	NULL
167 user_123	"	919 bytes
168 user_123	"	52 bytes
174 user_123	"	NULL

3. 장기 기억을 위한 메모리 최적화

- LLM은 한 번에 처리할 수 있는 정보의 양, 즉 문맥 윈도우(context window)에 제한이 있음
- API 사용료는 입력되는 토큰의 양에 비례하여 부과됨
- 사용자와의 대화가 수백, 수천 턴(turn) 이상 이어진다면, 수천 마디 대화 기록을 모두 포함해서 LLM에 전송해야 하므로 비용이 어마어마하게 늘어나게 됨
- 또한, 너무 많은 정보는 오히려 LLM의 판단력을 흐리게 하거나, 중요한 최신 정보를 잊게 만들 수 있고, 최악의 경우에는 모델이 허용하는 최대 토큰 수를 초과하여 오류가 발생하고 에이전트가 멈추게 됨
- 이 문제의 해결 방법
 - 최근 대화만 원본 그대로 유지하고 오래된 대화는 '핵심 내용만 요약(summarization)'하여 저장

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 랭그래프와 에이전트 시스템에서는 대화가 일정 길이를 넘어가면 자동으로 요약을 수행해 주는 미들웨어(middleware)를 사용할 수 있음
- 대화가 진행될 때마다 현재 누적된 토큰 수를 확인하고, 설정해 둔 기준을 초과하면, LLM에게 “지금까지의 대화를 요약해 줘”라고 요청하는 방식임

- 기본 설정

- 이번 실습은 ‘summarization_agent.py’ 파일에서 진행

```
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_openai import ChatOpenAI
import sqlite3
from dotenv import load_dotenv

load_dotenv()
model = ChatOpenAI(model="gpt-5-nano", temperature=0)

# 데이터베이스 연결 및 SqliteSaver 초기화
conn = sqlite3.connect("memory.db", check_same_thread=False)
checkpointer = SqliteSaver(conn)
```


3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 미들웨어 설정

```
from langchain.agents.middleware import SummarizationMiddleware
from langchain.agents import create_agent

# 1. 요약 미들웨어 설정
summarization_mw = SummarizationMiddleware(
    model=model,
    trigger=[('tokens', 500)],
    keep=('messages', 5)
)
```

- SummarizationMiddleware 클래스로 요약 미들웨어 객체 색성
 - 첫 번째 인자는 대화를 요약할 모델 선택
 - 요약 미들웨어의 핵심은 대화를 ‘요약’하는 것
 - 이 요약 작업 역시 LLM의 힘을 빌림
 - 긴 문서를 읽고 핵심에 집중해 요약하는 것은 LLM이 매우 잘 하는 일 중 하나
 - 두 번째 인자는 제한할 토큰의 수
 - 요약 미들웨어가 언제 업무를 수행할지(trigger) 결정
 - 테스트를 위해 500개의 아주 적은 양의 토큰으로 설정하겠음
 - 토큰 계산 방법은 오픈AI의 토큰 계산 방식을 따름

3. 장기 기억을 위한 메모리 최적화

● SummarizationMiddleware로 대화 요약하기

- 미들웨어 설정
 - 토큰 계산 방식은 옆 그림과 공식 문서 참조
 - <https://platform.openai.com/tokenizer>
 - 세 번째 인자는 유지할 맥락의 수
 - 요약을 마친 후, 얼마나 많은 기록을 보존할지(keep) 지정
 - 기본값은 가장 최근의 20개 메시지를 요약하지 않고, 그대로 유지하는 것.
 - 이번 실습에는 확인을 위해 5개의 메시지만 유지하도록 설정하겠음
- trigger와 keep 두 인자에 값을 전달하는 방법은 기본적으로 튜플 형식을 취함
- 각 인자에 전달하려는 설정이 여러 개의 튜플이라면 리스트로 묶어서 한 번에 전달

The screenshot shows the OpenAI Tokenizer Playground interface. At the top, there are three tabs: "GPT-5.x & O1/3", "GPT-4 & GPT-3.5 (legacy)", and "GPT-3 (legacy)". The first tab is selected. Below the tabs is a text input area containing the text: "안녕하세요. 반갑습니다. 저는 현재 langchain의 agent를 구현하는 방법을 공부하고 있습니다." Below the text input are two buttons: "Clear" and "Show example". Below these buttons are two columns of text: "Tokens" and "Characters". Under "Tokens" is the number "22", and under "Characters" is the number "57". Below these numbers is a text area showing the input text with individual tokens highlighted in different colors. At the bottom, there are two buttons: "Text" and "Token IDs".

GPT-5.x & O1/3 GPT-4 & GPT-3.5 (legacy) GPT-3 (legacy)

안녕하세요. 반갑습니다. 저는 현재 langchain의 agent를 구현하는 방법을 공부하고 있습니다.

Clear Show example

Tokens Characters

22 57

안녕하세요. 반갑습니다. 저는 현재 langchain의 agent를 구현하는 방법을 공부하고 있습니다.

Text Token IDs

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 에이전트 생성

```
# 2. Agent 생성 (미들웨어 포함)
agent = create_agent(
    model=model,
    checkpointer=checkpointer,
    middleware=[summarization_mw]
)
```

- 앞서 생성한 미들웨어도 함께 전달
 - 미들웨어 역시 도구와 같이 여러 종류를 설정하고, 전달할 수 있으므로 리스트에 담아 전달

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 동작 테스트

```
# 3. 사용자 설정
user_id = "user_summary"
config = {"configurable": {"thread_id": user_id}}

# 연속 질문 리스트
queries = [
    "AI의 역사에 대해 간략히 설명해줘.",
    "머신러닝과 딥러닝의 차이점은 뭐야?",
    "트랜스포머 모델의 핵심 원리가 뭐지?",
    "BERT 모델은 어떻게 학습해?",
    "GPT 모델의 특징은 무엇이냐?",
    "자연어 처리에서 주로 사용하는 데이터셋에는 어떤 것들이 있어?",
    "AI 윤리 문제에 대해 설명해줄 수 있어?",
    "미래의 AI 기술 발전 방향에 대해 어떻게 생각해?",
    "파이썬의 기초 문법에 대해서 알려줄래?",
    "웹 개발에서 프론트엔드와 백엔드의 차이는 뭐야?",
    "데이터베이스의 정규화란 무엇인가?",
    "클라우드 컴퓨팅의 장점은 뭐야?",
    "컨테이너화 기술에 대해 설명해줘.",
    "마이크로서비스 아키텍처의 특징은 뭐지?",
    "지금까지 우리가 무슨 이야기를 했는지 요약해줄래?"
]
```

```
# 4. 모든 질문에 대해서
for index, query in enumerate(queries, 1):
    print(f"\n[{index}] 사용자: {query}")

    for chunk in agent.stream(
        {"messages": [{"role": "user", "content": query}]},
        config,
        stream_mode="values"
    ):
        # 각 chunk는 해당 시점의 전체 state를 포함
        messages = chunk["messages"]
        print(f"메시지 개수: {len(messages)}")

# 마지막 메시지 기준 요약 메시지 확인
for msg in messages:
    if "summary" in str(msg).lower():
        print(f"\n{msg.content}")
```

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 동작 테스트

- 이제 대화를 길게 이어가며 실제로 요약이 발생하는지 확인
- 이전 대화와 구분 짓기 위해 thread_id를 “user_summary”로 설정하고, 여러 가지 질문을 연속으로 던져 토큰 수를 의도적으로 늘려 보겠음
- 질문의 내용은 최소 10개 이상 작성
- 에이전트의 호출 흐름을 확인하기 위해 stream 메서드 사용
- stream_mode를 “values”로 설정하면 이번 호출에 몇 개의 대화 맥락이 사용되었는지 모두 확인할 수 있음
- 대신, 에이전트의 실제 답변이 중요한 것은 아니므로 전체 대화 맥락의 메시지 수만 체크하도록 하겠음

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 출력 결과 확인

- 반복문을 통해 순차적으로 질문이 진행되면서 차근차근 메시지가 쌓여가는 것을 볼 수 있음
- 출력문을 통해 요약이 진행되고 있음을 유추할 수 있음
- 질문마다 남은 메시지의 수가 7개인 이유는, 유지한 메시지 5개와 이번 질문과 답변 2개를 합한 수
- 이번 실습에서 토큰이 500을 넘을 때마다 요약을 진행하도록 했는데, 이는 아주 적은 수임
- 대화를 요약하더라도, 요약된 정보 역시 토큰을 차지하므로 이후의 모든 질문에 미들웨어가 동작한 상태에서 메시지를 지워 7개만 남기는 모습을 보임

[1] 사용자: AI의 역사에 대해 간략히 설명해줘.

메시지 개수: 8

메시지 개수: 6

메시지 개수: 7

... (생략) ...

[15] 사용자: 지금까지 우리가 무슨 이야기를 했는지 요약해줄래?

메시지 개수: 8

메시지 개수: 6

메시지 개수: 7

3. 장기 기억을 위한 메모리 최적화

● SummarizationMiddleware로 대화 요약하기

• 출력 결과 확인

Here is a summary of the conversation to date:

- 파이썬 학습 컨텍스트

- 초보자 관점의 파이썬 기초 정리 제안: 설치/실행, 기본 문법, 기본 연산자, 제어 흐름, 데이터 구조, 문자열, 함수, 모듈/패키지, 입출력, 예제 실습, 주의점/팁.
- 프론트엔드와 백엔드 차이 및 데이터 흐름 개요: 프론트엔드/UI 중심 vs 백엔드 인증/데이터 저장/비즈니스 로직/API 엔드포인트.
- 데이터 흐름: 프론트엔드 → 백엔드 API → DB 조회/갱신 → 프론트엔드 표시; RESTful API, GraphQL, 서버리스/마이크로서비스 가능 성.
- 보안/품질 포인트: 서버 측 검증, 인증 방식(JWT vs 세션 쿠키), 데이터 검증, 프론트엔드/백엔드 테스트의 중요성.
- 팀 역할 및 시작 포인트: 프론트엔드→백엔드→풀스택 순으로 시작; 맞춤 로드맵으로 쇼핑몰/블로그/간단한 게임 등 예제 제안 가능.
- 맞춤 학습 로드맵/예제 제안 가능성: 관심 분야나 목표를 알려주면 구체적 로드맵 및 예제 제공 제안.

- 데이터베이스 정규화 컨텍스트

- 정규화 정의: 데이터 중복 최소화과 무결성 향상을 위한 테이블 분리 및 속성 간 종속성 정리.
- 주요 목표: 기능적 종속성 분석을 통해 비주요 속성이 후보키에 의존하도록 설계; 중복 제거로 업데이트 이상 현상 방지.
- 정상형 요약: 1NF, 2NF, 3NF, BCNF, (필요 시 4NF/5NF) 및 현실적 적용의 트레이드오프(조인 비용 증가/비정규화로 인한 성능 고려).
- 간단한 예시: 학생ID/과목ID 기반 초기 테이블에서 1NF 충족 및 2NF/3NF로의 분해 예시 제시.
- 실무 포인트: 테이블 구성의 적절성, 이행 종속의 제거 여부, 외래키 제약, 데이터 무결성과 조회 성능 간 균형.
- 워크플로우: 요구사항 분석 → 엔터티-관계 모델 → 논리 모델(정규화) → 물리 모델(인덱스/파티셔닝) 구현.
- 제안: 구체 스키마를 주면 1NF부터 3NF/BCNF까지 단계별 정규화와 각 단계의 분해 근거를 자세히 설명하고, 도메인 예시(온라인 쇼핑, 도서관 시스템 등)로 시연 가능.
- 추가 안내: 과도한 정규화의 성능 영향 및 필요 시 비정규화 고려 가능성.

continue

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 출력 결과 확인

- 클라우드 컴퓨팅의 장점

- 비용 효율성 및 자본 비용 감소: 초기 CAPEX 없이 필요 시 자원을 사용하고 PAYG.
 - 확장성(스케일링)과 탄력성: 트래픽 증가 시 자원 자동 확장, 수요 감소 시 축소.
 - 가용성 및 재해복구: 다중 리전/가용영역 데이터 복제와 자동 백업/복구.
 - 접근성 및 전세계 배포: 인터넷 연결만으로 어디서나 접근 가능, 글로벌 사용자에게 대한 낮은 지연.
 - 개발 속도와 운영 효율성: 인프라 관리 부담 감소, CI/CD/프로비저닝 자동화 용이.
 - 최신 기술 접근성: 머신러닝, 빅데이터, 서버리스 등 최신 도구를 손쉽게 이용.
 - 보안 및 규정 준수 도구의 활용: IAM, 암호화, 감사 로그 등 기본 제공 또는 구성 용이.
 - 운영 비용 예측 가능성 및 관리 용이성: 사용량 기반 과금, 모니터링/알림/자동화 도구.
 - 간단한 예시: 스타트업이 소량의 서버로 시작해 수요 증가 시 자동 확장/글로벌 확장으로 품질 유지.
 - 도메인 맞춤 사례/로드맵 제안 가능 여부: 어떤 분야를 목표로 하는지에 따라 구체 사례 제공 가능.

- 사용자의 의도

- 클라우드 컴퓨팅의 장점에 대한 요약과 도메인 맞춤 사례/로드맵 제안 요청.

- 마지막 메시지 기준 요약 메시지 확인을 위해 출력할 수 있도록 코드를 작성함
 - 그 결과, “Here is a summary of the conversation to date.”라는 텍스트와 함께 각 질문들에 대한 요약문이 출력됨
 - 이 문구의 경우, LLM의 특성상 차이가 있을 수 있음
 - 중요한 것은, 전체 대화의 내용을 기억하고 충분히 요약하여 답변할 수 있는 능력이 있다는 점

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

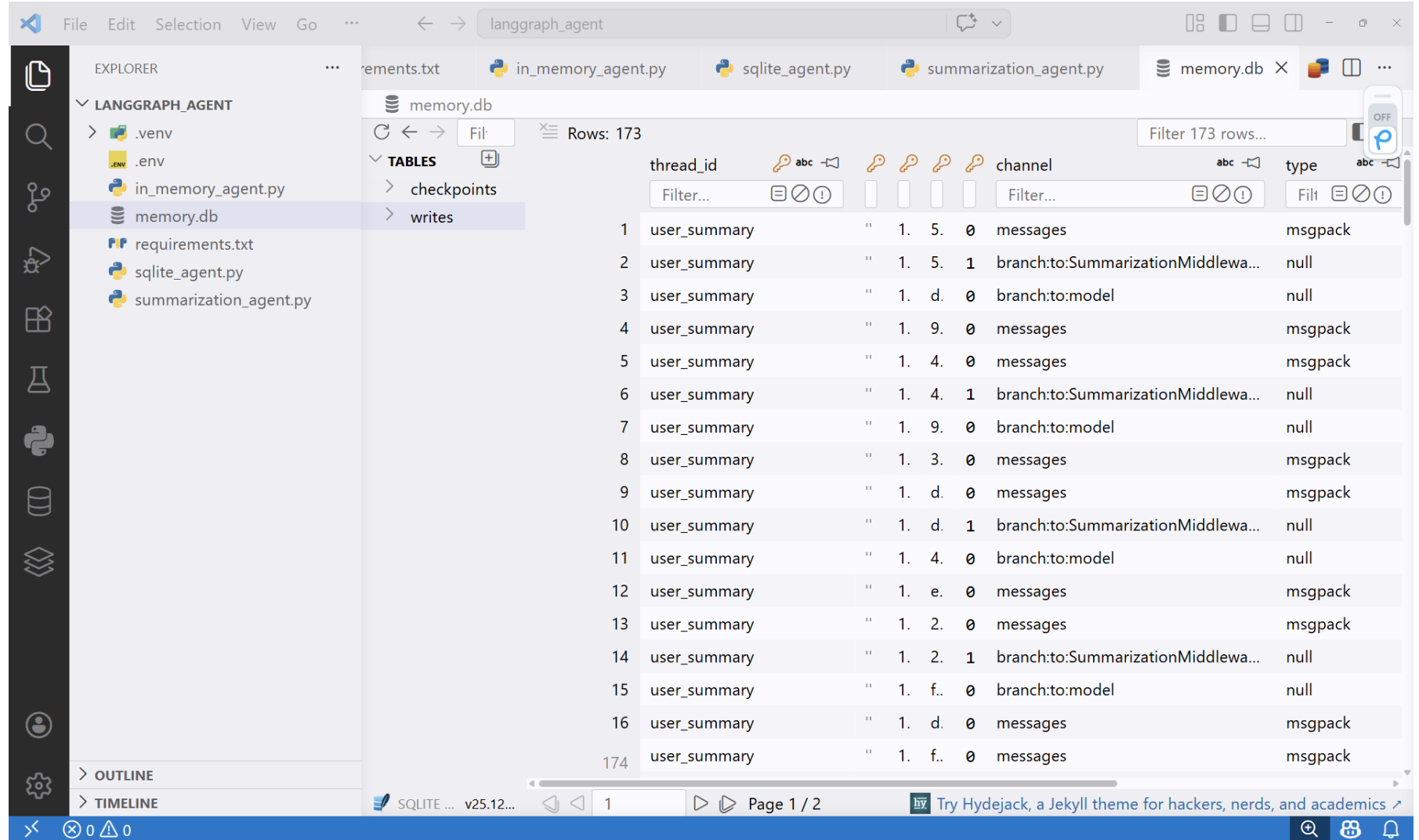
- 데이터베이스 확인

- 마지막으로 확인할 것은 ‘이 전체 과정이 정말 SummarizationMiddleware를 통해서 진행되었는가’라는 사실
- 이미 우리가 사용하고 있는 LLM들은 텍스트를 충분히 요약해서 제공할 수 있음
- 가장 확실한 확인 방법은 데이터베이스를 확인해 보는 것
- 총 15개의 질문에 대해 146개의 행이 저장되어 있음(이 결과는 LLM 모델의 응답에 따라 당연히 모두 다를 수 있음)
- ‘사용자 질문 -> 미들웨어 확인 -> 모델 호출 -> 모델의 답변’ 순으로 작업이 진행된 것임
- 미들웨어가 동작한 것은 다음 페이지의 writes 테이블의 channel 컬럼에 저장된 미들웨어 관련 행을 볼 수 있음
- 요약 미들웨어가 메시지를 줄인다고 했는데 행이 왜 이렇게 많을까?
 - 미들웨어의 역할을 생각해 보면 이해하기 쉬움
 - 요약 미들웨어의 역할은 데이터베이스의 정리가 아님
 - 데이터베이스에는 여전히 모든 대화 내역이 저장되어 있어야 함
 - 미들웨어가 하는 일은 어디까지나, 이전까지의 대화 내용을 요약하고 모델에게 전달할 메시지의 수를 줄이는 것
 - checkpoints 테이블 역시 마찬가지임
 - 앞서 언급한 실행 순서 ‘사용자 질문 -> 미들웨어 확인 -> 모델 호출 -> 모델의 답변’의 모든 과정은 하나의 ‘체크포인트’임
 - 각 행동이 실행된 모든 상태는 데이터베이스에 저장되고, 이를 요약 미들웨어가 삭제할 권한은 없음

3. 장기 기억을 위한 메모리 최적화

- SummarizationMiddleware로 대화 요약하기

- 데이터베이스 확인



The screenshot shows a VS Code editor with a SQLite database named 'memory.db' open. The database contains a table with 173 rows. The table has columns for 'thread_id', 'user_summary', 'channel', and 'type'. The data shows a sequence of messages and summaries for a single thread (thread_id 1).

thread_id	user_summary	channel	type
1	user_summary	messages	msgpack
2	user_summary	branch:to:SummarizationMiddlewa...	null
3	user_summary	branch:to:model	null
4	user_summary	messages	msgpack
5	user_summary	messages	msgpack
6	user_summary	branch:to:SummarizationMiddlewa...	null
7	user_summary	branch:to:model	null
8	user_summary	messages	msgpack
9	user_summary	messages	msgpack
10	user_summary	branch:to:SummarizationMiddlewa...	null
11	user_summary	branch:to:model	null
12	user_summary	messages	msgpack
13	user_summary	messages	msgpack
14	user_summary	branch:to:SummarizationMiddlewa...	null
15	user_summary	branch:to:model	null
16	user_summary	messages	msgpack
174	user_summary	messages	msgpack